Cynbit Backend Developer Intern Assessment

Candidate: Umer Rafiq
Assessment date: 22–23 October 2026 (IST)
Target: `https://www.cynbit.in/`
Method: Passive review of public pages, ordinary HTTPS GETs, browser observations, and static inspection of publicly referenced JavaScript.

> **Scope note:** The local FastAPI API and contact-form state model in this repository are recommended reference implementations. They are not Cynbit source code, do not connect to Cynbit, and do not demonstrate its backend behavior.

1. Executive summary

Eight public routes returned HTTP 200 in the latest sampled GET pass. All returned `Strict-Transport-Security: max-age=63072000`. Five selected browser-hardening headers were absent from those responses. Wildcard CORS appeared on five public HTML routes; no sensitive response or authenticated endpoint was tested, so this is not a CORS vulnerability finding.

One **low-severity frontend reliability defect (REL-01)** is supported by the current public contact bundle's control flow: a rejected `submitInquiry` Promise bypasses the pending-state reset, leaving the submit button disabled. This is a verified code-path defect, **not a live failure reproduced on Cynbit**. The local reference model implements a `finally` reset and its tests verify recovery and retry.

No exploitable security vulnerability, server-side defect, or privacy-law violation was confirmed. Applicant fields and browser-storage names warrant owner review, but their necessity, source, and processing are unknown. Neither form was submitted. The local API example passes 23 tests; its independent Node retry model passes 3 tests.

2. Scope, methodology, and authorization boundaries

Included: home, careers, backend-internship detail, services, contact, privacy, cookies, and terms pages; rendered forms and page text; selected response headers; referenced contact and application JavaScript bundles; browser storage names without values; one console observation on `/cookies`.

Excluded: form submissions, invalid-input testing on production, authentication/private resources, scanning, fuzzing, load testing, source/infrastructure/database access, and production data changes. No credentials were used. Requests were ordinary GETs; static bundles were fetched only when referenced by public pages. The browser did not provide a complete Network export or cookie-attribute inspection.

The header pass used PowerShell `Invoke-WebRequest -UseBasicParsing` with default redirect handling. Reported status/header values are from the final response; redirect chains and intermediate headers were not recorded. No live server-action request/response was captured. Detailed observations and earlier-pass evidence are in the verification log.

3. Public routes and evidence reviewed

Full URL	Final GET result	Key response headers (23 Oct 2026, IST)	Evidence
`https://www.cynbit.in/`	200, HTML, 19:26:42	HSTS `max-age=63072000`; CORS `*`; selected five headers absent; no `Set-Cookie`	E-001, E-018
`https://www.cynbit.in/careers`	200, HTML, 19:26:43	HSTS; no CORS; selected five headers absent; no `Set-Cookie`	E-002, E-018
`https://www.cynbit.in/careers/backend-developer-intern`	200, HTML, 19:26:45	HSTS; no CORS; selected five headers absent; no `Set-Cookie`	E-007, E-008, E-018
`https://www.cynbit.in/services`	200, HTML, 19:26:46	HSTS; CORS `*`; selected five headers absent; no `Set-Cookie`	E-003, E-018
`https://www.cynbit.in/contact`	200, HTML, 19:26:46	HSTS; no CORS; selected five headers absent; no `Set-Cookie`	E-004, E-009, E-018
`https://www.cynbit.in/privacy`	200, HTML, 19:26:46	HSTS; CORS `*`; selected five headers absent; no `Set-Cookie`	E-011, E-018
`https://www.cynbit.in/cookies`	200, HTML, 19:26:46	HSTS; CORS `*`; selected five headers absent; no `Set-Cookie`	E-012, E-018
`https://www.cynbit.in/terms`	200, HTML, 19:26:46	HSTS; CORS `*`; selected five headers absent; no `Set-Cookie`	E-013, E-018

“Selected five” means CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy. Earlier one-off HTML sizes are recorded in the evidence inventory; they are not performance benchmarks. E-019 records the public contact bundle GET (200, 24,534 bytes, 23 Oct 19:24:38 IST). E-014 records the earlier contact/application bundle observations.

4. Backend architecture and observable behavior

The homepage exposed `Server: Vercel`; public resources used `/_next/static/...` assets and Next.js Server Reference helpers. The contact bundle references `submitInquiry`, creates `FormData`, and displays a success message claiming the inquiry was received and architects notified. The application bundle references `submitCandidateApplication` and adds a job identifier and serialized education data.

These observations establish the shipped client flow only. No form action was invoked; the actual request method/payload, server validation, persistence, notification delivery, duplicate handling, authorization, and backend architecture remain unknown. A third-party analytics POST appeared as `ERR_ABORTED` during navigation in the limited browser event view; this does not establish an application failure.

The contact form displayed five text-entry controls. The internship form displayed 23 controls and required fields including date of birth, gender, phone, WhatsApp, and permanent address. The public privacy notice describes typical collection such as name, email, and professional context and names Resend and Prisma among service providers; the cookie notice describes Google Analytics/page-interaction metrics. These are public statements, not verified implementation details. Browser storage names observed in a shared browser context were `_ga`, `_ga_LK6CQWCWX0`, `_gcl_au`, `_gcl_ls`, and `cynbit_newsletter_seen`; no values were read. Their setter, age, purpose, attributes, and consent state are unknown.

5. Prioritized findings

ID	Severity / confidence	Type	Finding and evidence
REL-01	**Low** / 9 of 10 for static code path	Verified client-code defect; runtime failure not reproduced	Rejected contact action bypasses pending reset; submit button remains disabled. E-019; local fix/model E-020.
HDR-01	**Informational** / 9 of 10	Hardening observation, not an exploit	Five selected headers absent on the eight sampled HTML responses; exact routes/statuses in E-018.
PRIV-01	**Potential review item; no vulnerability severity** / 9 of 10 for visible fields	Data minimization/transparency	Required applicant details exceed the privacy notice's examples; purpose, notice applicability, retention, and handling are unverified. E-008, E-011.
PRIV-02	**Potential review item; no vulnerability severity** / 9 of 10 for names; 5 of 10 for any disclosure gap	Storage disclosure	Cookie/local-storage names were observed but their source, age, purpose, consent state, and values were not established. E-012, E-015.
CORS-01	**Informational** / 9 of 10 for sampled responses	Header observation only	`Access-Control-Allow-Origin: *` on home, services, privacy, cookies, and terms. These sampled responses contained public HTML. No credentialed or sensitive API response was tested. E-018.

No confirmed security vulnerability: Header absence alone does not establish XSS or clickjacking; wildcard CORS on public HTML alone does not establish data exposure. Authentication, authorization, API validation, rate limits, database controls, and server-side error handling were not verified and are not reported as defects.

Proportionate follow-up and validation:

- **HDR-01:** Review actual frontend resources/embeds with the owner, deploy compatible headers in staging, start CSP in report-only mode, then verify all intended pages and API responses. Framing-policy compatibility needs a product/embed check.
- **PRIV-01:** Ask the recruitment/data owner for the purpose and collection stage of each required field, then compare that documented purpose with the applicant notice. In authorized staging/design documentation, verify retention/deletion, recruiter roles, and that body fields are excluded from logs.
- **PRIV-02:** In a clean authorized browser profile, record storage names and attributes before/after consent and compare with tag-manager/vendor configuration and the public notice. The existing shared-profile names alone are insufficient to establish site ownership or disclosure gaps.
- **CORS-01:** Review intended cross-origin clients and API configuration; only assess browser access to any non-public response in an authorized staging/account context. Do not infer risk from public HTML alone.

HDR-01: practical interpretation

- HSTS was present with `max-age=63072000`. The observed value did not include `includeSubDomains` or `preload`; desired subdomain scope was not assessed.
- CSP may reduce impact of some injection bugs but must account for Next.js scripts, styles, fonts, analytics, and embeds. Pilot in report-only mode.
- `X-Content-Type-Options: nosniff` is generally compatible when content types are correct.

- `X-Frame-Options: DENY` or CSP `frame-ancestors` can reduce clickjacking risk, but may prevent intentional embedding; confirm requirements.
- A policy such as `Referrer-Policy: strict-origin-when-cross-origin` can reduce referrer leakage. Permissions-Policy should disable only unused browser capabilities.
- Validate chosen policies on real page and API routes in an authorized deployment; do not blindly copy an API-only policy to the frontend.

6. REL-01: contact-form state-path analysis

The latest bundle initializes pending/success/error state as `[e,t]`, `[a,i]`, and `[l,n]`. Handler `d` prevents default, calls `t(true)`, clears the prior error, constructs `FormData`, awaits action reference `x` (`submitInquiry`), then calls `t(false)` before checking the resolved result. The submit button has `disabled:e`.

Path	Observed control flow
Resolved success	Pending resets false; success view is shown.
Resolved `{success: false}`	Pending resets false; a retryable error is displayed.
Rejected Promise / thrown operation before reset	Control exits before `t(false)`; pending stays true and the button remains disabled. No local catch/finally exists.

This is a **verified code-path defect** because the pending state is directly bound to the button and all state-changing operations before reset are capable of throwing. It is **not evidence that the live site experienced such a rejection**. No form was submitted and no production failure was induced.

Minimal recommended handler shape:

```
async function handleSubmit(event) {
  event.preventDefault();
  setPending(true);
  setError(null);

  let result;
  try {
    result = await submitInquiry(new FormData(event.currentTarget));
  } catch {
    setError("Something went wrong. Please try again or email us directly.");
    return;
  } finally {
    setPending(false);
  }

  if (!result?.success) {
    setError("Something went wrong. Please try again or email us directly.");
    return;
  }
  setSuccess(true);
}
```

The independent local state model follows this path; Node tests cover resolved success, resolved application failure, rejected Promise, restored pending state, generic error text, and a successful retry. It is a demonstration, not a patch to Cynbit's deployed frontend.

7. Recommended backend improvements and trade-offs

For any real form receiver, independently validate on the server even if the browser uses `required` or HTML constraints. Normalize surrounding whitespace, enforce field/body limits, reject unexpected fields, return stable field-level errors without echoing user values, and log request IDs/status/timing rather than form bodies.

The local FastAPI API demonstrates Pydantic `EmailStr`, bounded fields, whitespace normalization, `extra="forbid"`, a consistent JSON error envelope, request IDs, generic internal errors, privacy-conscious logs, and a configurable body cap (`CYNBIT_REFERENCE_MAX_BODY_BYTES`, default 16 KiB). The body cap handles declared and streamed bodies. API security headers are restricted to JSON routes; interactive docs are excluded from the API-only CSP. HSTS belongs at the trusted HTTPS edge.

Representative server-side model:

```
class InquiryRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    name: str = Field(min_length=1, max_length=100)
    email: EmailStr
    organization: str | None = Field(default=None, max_length=150)
    industry: str | None = Field(default=None, max_length=100)
    message: str = Field(min_length=10, max_length=2000)

    @field_validator("name", "organization", "industry", "message", mode="before")
    @classmethod
    def strip_text(cls, value: object) -> object:
        return value.strip() if isinstance(value, str) else value
```

The reference API deliberately stores and forwards nothing, and has no database, authentication, CORS policy, or rate limiter. A real staff/admin function would require authentication and authorization; a public form may need edge abuse controls, but thresholds should follow measured traffic. If adding email delivery, configure bounded timeouts and appropriate retry/idempotency behavior. Enforce request size at both edge and application layers.

For recruitment data, confirm purpose for each required field, make fields optional/progressive where appropriate, state access and retention/deletion rules, restrict recruiter access by role, and ensure applicant/contact data is excluded from logs. The observed UI and policy text do not establish current backend practices or legal compliance.

8. Testing and verification

Command	Result
`python -B -m pytest -p no:cacheprovider`	Exit 0; **23 passed** (Python 3.11.9, pytest 9.1.1). Covers validation, errors, logs, request IDs, 404/405, declared/streamed body limits, invalid config, configured limit, and API headers.
`node --test code-examples/tests/contact-form.test.mjs`	Exit 0; **3 passed** (Node v24.15.0). Covers success, resolved failure, rejected Promise, pending reset, generic error, and retry.
`python -m pip install --disable-pip-version-check -r requirements.txt` (fresh Python 3.11 venv)	Exit 0; exact pins installed.
`python -m pip check` (fresh venv)	Exit 0; no broken requirements.

Command	Result
Local Uvicorn CLI launch (`python -m uvicorn main:app --app-dir code-examples\app --host 127.0.0.1 --port 8767`)	PowerShell smoke script exited 0; `GET /health` returned 200 `{"status":"ok"}`. The test PID was explicitly stopped after the response; no graceful server exit code is claimed. Auto-reload was disabled.
Python AST parsing and app import	Passed; routes `/health`, `/inquiries`, and local OpenAPI docs loaded.
Markdown-to-PDF export and inspection	Regenerated from this report; 6 pages. Extracted text contains all 10 sections, all 5 finding IDs, current test counts, and page footers; visually checked in the integrated PDF reader.

These are local example tests only and do not test Cynbit's production service. The verification log records the prior review, current evidence IDs, limitations, and test results.

9. Limitations and authorized follow-up

- No first-party server-action/API request or response was captured. No contact or application form was submitted; no production records were created.
- Server-side validation, request limits, rate limits, persistence, notifications, authentication/authorization, CSRF applicability, and access/retention controls remain unverified.
- Browser cookie-attribute inspection was unavailable. Only storage names were observed; no values were read. A single reload of `/cookies` produced no observed console/page errors; this is not a site-wide check.
- Privacy/cookie page claims were not independently verified against vendor configurations or internal data flows. No legal assessment was performed.
- No repeatable performance test, full HAR/Network export, redirect-chain capture, API inventory, or deployment-config review was performed.

Any submission testing requires written authorization and a designated staging environment or approved test record. A clean browser profile and site-owner tag/configuration review are needed to establish cookie setters, purposes, consent timing, and retention.

10. Conclusion and prioritized next steps

The strongest concrete result is REL-01, a low-severity, statically verified client reliability defect with a local tested fix; its live failure path remains untested. The remaining web findings are hardening observations and privacy-review questions, not confirmed vulnerabilities. No high- or critical-severity issue is supported by the available evidence.

1. If the site owner confirms the bundle is current, fix and stage-test the pending-state `finally` behavior.
2. With authorization, capture sanitized staging form requests/responses and verify server validation, body limits, abuse controls, data handling, and error behavior.
3. Reconcile recruitment fields and browser-storage identifiers with purpose-specific notices, consent, access, and retention policies.
4. Review header policies with frontend owners and test CSP in report-only mode before enforcement.
5. Gather repeatable performance telemetry before making scaling recommendations.

No repository commit, GitHub publication, or assessment submission has been made.